

Moduli e libreria standard

Date e orari

Lavorare con date e orari usando il modulo `datetime` in Python.

Perché è utile usare date e orari in modo corretto

Qualunque programma reale prima o poi ha a che fare con le date: "quando è stata inviata questa email?", "quanti giorni mancano al compleanno?", "quanti giorni fa ho creato questo file?". Python ha un modulo dedicato per tutto questo.

Il modulo `datetime`

Il modulo `datetime` offre quattro strumenti principali:

Classe	Cosa rappresenta
<code>date</code>	Solo la data (anno, mese, giorno)
<code>time</code>	Solo l'orario (ore, minuti, secondi)
<code>datetime</code>	Data e orario insieme
<code>timedelta</code>	Una durata — la differenza tra due momenti

```
from datetime import date, datetime, timedelta # Importa solo quello che serve
```

La data e l'ora di adesso

```
from datetime import date, datetime

oggi = date.today()      # Solo la data odierna
print(oggi)              # 2024-01-15 (formato: anno-mese-giorno)

adesso = datetime.now()  # Data e orario attuali
print(adesso)           # 2024-01-15 14:30:25.123456
```

Creare una data specifica

```
from datetime import date, datetime

compleanno = date(2008, 3, 25)      # anno=2008, mese=3, giorno=25
print(compleanno)                  # 2008-03-25

capodanno = datetime(2025, 1, 1, 0, 0, 0) # anno, mese, giorno, ora,
                                         # minuti, secondi
print(capodanno)                   # 2025-01-01 00:00:00
```

Accedere alle singole parti di una data

```
from datetime import datetime

adesso = datetime.now()

print(adesso.year)      # Anno (es: 2024)
print(adesso.month)     # Mese (1-12)
print(adesso.day)       # Giorno (1-31)
print(adesso.hour)      # Ora (0-23)
print(adesso.minute)    # Minuti (0-59)
print(adesso.second)    # Secondi (0-59)
```

Formattare una data come testo: `strftime()`

Per default le date si stampano nel formato `2024-01-15`. Con `strftime()` puoi scegliere tu come vuoi che appaia:

```
from datetime import datetime

adesso = datetime.now()

print(adesso.strftime("%d/%m/%Y"))      # 15/01/2024 (formato italiano)
print(adesso.strftime("%d %B %Y"))      # 15 January 2024 (con nome del
mese)
print(adesso.strftime("%H:%M:%S"))      # 14:30:25 (solo l'orario)
print(adesso.strftime("%d/%m/%Y %H:%M")) # 15/01/2024 14:30
```

I codici di formato principali:

Codice	Significato	Esempio
%Y	Anno (4 cifre)	2024
%m	Mese (2 cifre)	01
%d	Giorno (2 cifre)	15
%H	Ora in formato 24h	14
%M	Minuti	30
%S	Secondi	25
%A	Nome del giorno (inglese)	Monday
%B	Nome del mese (inglese)	January

Convertire testo in data: `strptime()`

L'operazione inversa: prendi una stringa di testo e trasformala in un oggetto data. Devi dirle in che formato è scritta la stringa:

```
from datetime import datetime

testo = "25/03/2008"
data = datetime.strptime(testo, "%d/%m/%Y") # Dici a Python come è
formattata la stringa
print(data)          # 2008-03-25 00:00:00
print(data.year)     # 2008
```

Calcolare differenze tra date: `timedelta`

`timedelta` rappresenta una **durata** — un certo numero di giorni, ore, minuti, ecc. Puoi aggiungerla o sottrarla a una data:

```

from datetime import date, timedelta

oggi = date.today()

# Tra quanti giorni sarà la prossima settimana?
tra_una_settimana = oggi + timedelta(days=7)
print(tra_una_settimana)

# Che giorno era 30 giorni fa?
trenta_giorni_fa = oggi - timedelta(days=30)
print(trenta_giorni_fa)

# Quanti giorni sono passati dal tuo compleanno?
nato = date(2008, 3, 25)
differenza = date.today() - nato      # Sottrai due date → ottieni un
timedelta
print(f"Giorni vissuti: {differenza.days}")

```

Confrontare date

Le date si confrontano come i numeri — con `<`, `>`, `==` :

```

from datetime import date

d1 = date(2024, 1, 1)
d2 = date(2024, 12, 31)

print(d1 < d2)      # True ← d1 viene prima di d2
print(d1 == d2)     # False ← sono date diverse

if date.today() > d1:
    print("Siamo dopo il 1 gennaio 2024")

```

JSON

Lavorare con il formato JSON in Python.

Qual è la funzione di JSON nelle applicazioni

Ogni volta che un'app sul tuo telefono mostra dati dal web — il meteo, i risultati di una ricerca, i post di un social network — quei dati spesso arrivano in formato **JSON**.

JSON è il "linguaggio comune" che i programmi usano per scambiarsi informazioni. Imparare a lavorare con JSON ti permette di fare cose concrete: scaricare dati da internet, salvarli, leggerli.

Cos'è il JSON?

JSON (si legge "djèizon") è un formato di testo per rappresentare dati strutturati. Assomiglia molto ai dizionari e alle liste Python:

```
{
  "nome": "Alice",
  "eta": 16,
  "voti": [8, 7, 9, 6],
  "indirizzo": {
    "citta": "Roma",
    "cap": "00100"
  },
  "attivo": true
}
```

È leggibile dagli esseri umani (se ben indentato) e facilmente interpretabile dai computer.

Importare il modulo

Da Python a JSON: `json.dumps()`

`dumps()` converte un dizionario (o una lista) Python in una **stringa** JSON. Utile per inviare dati o stamparli.

```
dati = {
    "nome": "Alice",
    "eta": 16,
    "voti": [8, 7, 9],
    "attivo": True,      # Python: True → JSON: true
    "media": None        # Python: None → JSON: null
}

# Converti in stringa JSON (su una riga)
json_stringa = json.dumps(dati)
print(json_stringa)
# → {"nome": "Alice", "eta": 16, "voti": [8, 7, 9], "attivo": true,
#   "media": null}

# Converti in stringa JSON leggibile (con indentazione)
json_bello = json.dumps(dati, indent=4)
print(json_bello)
```

Output indentato:

```
{
    "nome": "Alice",
    "eta": 16,
    "voti": [8, 7, 9],
    "attivo": true,
    "media": null
}
```

Conversione dei tipi: Python ↔ JSON

Python	JSON
<code>dict</code>	<code>{}</code> (oggetto)
<code>list</code>	<code>[]</code> (array)
<code>str</code>	<code>"..."</code> (stringa)
<code>int</code> , <code>float</code>	numero
<code>True</code> / <code>False</code>	<code>true</code> / <code>false</code>
<code>None</code>	<code>null</code>

Da JSON a Python: `json.loads()`

`loads()` fa il contrario: prende una stringa JSON e la converte in un dizionario (o lista) Python.

```
stringa_json = '{"nome": "Alice", "eta": 16, "voti": [8, 7, 9]}'

dati = json.loads(stringa_json)  # Converti la stringa in dizionario
Python

print(dati["nome"])             # Alice
print(dati["voti"])              # [8, 7, 9]
print(type(dati))                # <class 'dict'>
```

Scrivere JSON su file: `json.dump()`

`dump()` (senza la "s") scrive direttamente su un file:


```

studenti = [
    {"nome": "Alice", "eta": 16, "voto": 8},
    {"nome": "Bob", "eta": 15, "voto": 7},
]

with open("studenti.json", "w") as file:
    json.dump(studenti, file, indent=4) # Scrive il JSON nel file, ben
    indentato

```

Il file `studenti.json` conterrà:

```

[
  {
    "nome": "Alice",
    "eta": 16,
    "voto": 8
  },
  ...
]

```

Leggere JSON da file: `json.load()`

`load()` legge un file JSON e lo converte in Python:

```

with open("studenti.json", "r") as file:
    studenti = json.load(file) # Legge il file e converte in lista Python

for studente in studenti:
    print(f"{studente['nome']}: {studente['voto']}")
# → Alice: 8
# → Bob: 7

```

Gestire errori di formato

Se la stringa JSON non è valida (ad esempio ha una virgola di troppo), Python lancia un errore. Puoi gestirlo con `try/except` :

```
try:
    dati = json.loads("questo non è JSON valido!!!")
except json.JSONDecodeError as e:
    print(f"Errore: il testo non è JSON valido – {e}")
```

Esempio pratico: file di configurazione

Un uso comune di JSON è salvare le impostazioni di un programma in un file che l'utente può modificare:

```
# Salva le impostazioni
config = {
    "tema": "scuro",
    "lingua": "italiano",
    "notifiche": True,
    "volume": 75
}

with open("config.json", "w") as f:
    json.dump(config, f, indent=2)

# La prossima volta che il programma parte, ricarica le impostazioni
with open("config.json", "r") as f:
    config_caricata = json.load(f)

print(f"Tema: {config_caricata['tema']}")      # Tema: scuro
print(f"Lingua: {config_caricata['lingua']}") # Lingua: italiano
```

Matematica

Operazioni matematiche avanzate con il modulo math in Python.

Qual è la funzione del modulo math

Gli operatori base (`+` , `-` , `*` , `/` , `**`) coprono la maggior parte dei calcoli quotidiani. Ma appena ti servono radici quadrate, logaritmi, seno e coseno, o il valore di pi greco, hai bisogno del modulo `math` .

Per usarlo, basta questa riga all'inizio del tuo programma:

Costanti matematiche

Il modulo `math` include alcune costanti fondamentali già pronte. Eccole tutte:

```
print(math.pi)      # 3.141592653589793 – il pi greco (π)
print(math.e)        # 2.718281828459045 – il numero di Nepero (base dei
logaritmi naturali)
print(math.inf)      # inf – infinito (utile per confronti: nessun numero è
più grande)
print(math.nan)      # nan – "Not a Number", cioè "non è un numero" (risultato
di operazioni impossibili, come 0/0)
```

Arrotondamento

Queste funzioni arrotondano un numero decimale in modi diversi. Guarda come si comportano:

```
print(math.floor(3.7))    # 3 - arrotonda VERSO IL BASSO (piano inferiore)
print(math.ceil(3.2))     # 4 - arrotonda VERSO L'ALTO (piano superiore)
print(round(3.5))         # 4 - arrotondamento standard (già disponibile
                            senza math)
print(math.trunc(3.9))    # 3 - tronca la parte decimale (butta via i
                            decimali)
```

Analogia: pensa ai piani di un edificio. `floor` ti porta al piano di sotto, `ceil` a quello di sopra.

Valore assoluto

```
print(abs(-5))            # 5 - funzione già disponibile senza math
print(math.fabs(-5.0))    # 5.0 - versione di math (restituisce sempre un
                            decimale)
```

Il valore assoluto è la "distanza da zero" — elimina il segno negativo.

Potenze e radici

```
print(math.sqrt(16))      # 4.0 - radice quadrata ( $\sqrt{16} = 4$ )
print(math.sqrt(2))       # 1.4142135... - radice quadrata di 2

print(math.pow(2, 10))    # 1024.0 - potenza ( $2^{10}$ ), equivalente a  $2^{*}10$  ma
                            restituisce float
print(math.isqrt(17))     # 4 - radice quadrata intera (tronca, senza
                            decimali)
```

Logaritmi e funzioni esponenziali

```
# Esponenziale: e elevato alla x
print(math.exp(1))    # 2.718... - e^1
print(math.exp(2))    # 7.389... - e^2

# Logaritmo naturale (base e)
print(math.log(math.e))    # 1.0 - logaritmo naturale di e

# Logaritmo in base personalizzata
print(math.log(100, 10))   # 2.0 - log base 10 di 100 (quanto è 10^x = 100? Risposta: 2)
print(math.log10(1000))    # 3.0 - scorciatoia per log base 10
print(math.log2(8))        # 3.0 - scorciatoia per log base 2 (2^3 = 8)
```

Trigonometria

Le funzioni trigonometriche lavorano in **radianti**, non in gradi. Python include funzioni di conversione:

```
# Converti gradi ↔ radianti
print(math.radians(180))    # 3.14159... (180 gradi = π radianti)
print(math.degrees(math.pi)) # 180.0

# Seno, coseno, tangente
print(math.sin(math.pi / 2)) # 1.0 - seno di 90°
print(math.cos(0))          # 1.0 - coseno di 0°
print(math.tan(math.pi / 4)) # 1.0 - tangente di 45°
```

Altre funzioni utili

```
print(math.gcd(12, 8))      # 4   – Massimo Comune Divisore (MCD)
print(math.lcm(4, 6))      # 12  – Minimo Comune Multiplo (MCM) [Python
3.9+]
print(math.factorial(5))   # 120 – fattoriale: 5! = 5×4×3×2×1
```

Il modulo `random` : numeri casuali

Spesso usato insieme a `math`, il modulo `random` genera numeri casuali — utile per giochi, simulazioni, e molto altro:

```
print(random.random())      # Numero decimale casuale tra 0.0 e 1.0
print(random.randint(1, 10)) # Numero intero casuale tra 1 e 10
(inclusi)
print(random.uniform(1.0, 5.0)) # Numero decimale casuale tra 1.0 e 5.0

lista = [1, 2, 3, 4, 5]
print(random.choice(lista))  # Elemento scelto a caso dalla lista
random.shuffle(lista)        # Mescola la lista in ordine casuale
print(lista)                 # Esempio: [3, 1, 5, 2, 4]
```

Esempio pratico: calcoli geometrici

```
def area_cerchio(raggio):  
    # Formula:  $\pi \times r^2$   
    return math.pi * raggio ** 2  
  
def ipotenusa(cateto_a, cateto_b):  
    # Teorema di Pitagora:  $c = \sqrt{a^2 + b^2}$   
    return math.sqrt(cateto_a ** 2 + cateto_b ** 2)  
  
print(f"Area cerchio (r=5): {area_cerchio(5):.2f}")      # 78.54  
print(f"Ipotenusa (3, 4): {ipotenusa(3, 4):.2f}")        # 5.00
```

Moduli

Come organizzare il codice in moduli e importarli in Python.

Perché è utile usare i moduli

Immagina di dover scrivere un programma grande. Mettere tutto il codice in un unico file lungo migliaia di righe sarebbe un disastro — impossibile da leggere o da modificare.

I **moduli** sono il modo in cui Python ti permette di suddividere il codice in file separati, organizzati per argomento. Ogni file `.py` è già un modulo. E Python viene fornito con centinaia di moduli già pronti da usare — la cosiddetta **libreria standard**.

Importare un modulo con `import`

Per usare un modulo, devi "importarlo" all'inizio del tuo file:

```
print(math.pi)          # 3.141592653589793 – il pi greco
print(math.sqrt(16))    # 4.0 – radice quadrata
print(math.floor(3.7))  # 3 – arrotondamento verso il basso
print(math.ceil(3.2))   # 4 – arrotondamento verso l'alto
```

Quando scrivi `math.sqrt`, stai dicendo: "usa la funzione `sqrt` che si trova nel modulo `math`".

Importare solo quello che ti serve con `from ... import`

Se sai già quali funzioni ti servono, puoi importarle direttamente. Così non devi scrivere il nome del modulo ogni volta:

```
from math import pi, sqrt, floor # Importo solo queste tre cose

print(pi)          # 3.141592653589793
print(sqrt(25))    # 5.0
print(floor(4.9))  # 4
```

`from math import *` — sconsigliato

Si può importare tutto in una volta con `*`, ma è una cattiva abitudine:

```
from math import * # Importa TUTTO dal modulo math

print(sin(0))      # Funziona, ma...
```

Il problema: non si capisce più da dove vengono le funzioni. Se due moduli hanno una funzione con lo stesso nome, una sovrascrive l'altra senza avvertirti.

Rinominare con **as**

Puoi dare un soprannome a un modulo o a una funzione con la parola **as**. Utile se il nome è lungo o se vuoi evitare conflitti:

```
print(m.pi)          # 3.14159...

from math import sqrt as radice_quadrata  # Rinomino la funzione
print(radice_quadrata(9))  # 3.0
```

Alcune librerie molto usate hanno soprannomi convenzionali che tutti usano:

Creare il tuo modulo

Un modulo è semplicemente un file **.py**. Crea un file chiamato **calcolatrice.py**:

```
# calcolatrice.py – il mio modulo personale

def somma(a, b):
    return a + b

def sottrai(a, b):
    return a - b

def moltiplica(a, b):
    return a * b

pi = 3.14159
```

Poi in un altro file nella **stessa cartella**:

```
# main.py – uso il mio modulo

print(calcolatrice.somma(3, 4))    # 7
print(calcolatrice.pi)             # 3.14159
```

Il blocco `if __name__ == "__main__"`

Quando Python importa un modulo, esegue tutto il codice al suo interno. Questo può causare problemi se hai del codice di test che non vuoi far girare durante l'importazione.

La soluzione è racchiudere il codice di test in questo blocco speciale:

```
# calcolatrice.py

def somma(a, b):
    return a + b

if __name__ == "__main__":
    # Questo viene eseguito SOLO se avvii calcolatrice.py direttamente
    # NON viene eseguito se qualcuno fa "import calcolatrice"
    print(somma(3, 4))    # Test
```

Regola semplice: `__name__ == "__main__"` è `True` solo per il file che hai avviato tu, non per quelli importati.

Moduli utili già inclusi in Python

Modulo	A cosa serve
<code>math</code>	Funzioni matematiche (radici, trigonometria, logaritmi)
<code>random</code>	Numeri casuali
<code>datetime</code>	Date e orari
<code>os</code>	Interazione con il sistema operativo (file, cartelle)
<code>json</code>	Leggere e scrivere file JSON
<code>re</code>	Espressioni regolari (ricerche avanzate nel testo)
<code>time</code>	Misurare il tempo, mettere in pausa il programma

Il modulo `random` — numeri casuali

```
print(random.random())           # Numero decimale casuale
tra 0.0 e 1.0
print(random.randint(1, 10))     # Numero intero casuale tra
1 e 10 (inclusi)
print(random.choice(["mela", "banana", "uva"])) # Sceglie un elemento a
caso dalla lista

lista = [1, 2, 3, 4, 5]
random.shuffle(lista)            # Mescola la lista (modifica la lista originale)
print(lista)                     # Ordine casuale, es: [3, 1, 5, 2, 4]
```

Il modulo **OS** — interazione con il sistema

```
print(os.getcwd())          # Mostra in quale cartella ti trovi ora
print(os.listdir("."))      # Elenca i file nella cartella corrente
print(os.path.exists("file.txt")) # True se il file esiste, False
                                altrimenti
```

Espressioni regolari

Introduzione alle espressioni regolari con il modulo `re` in Python.

Perché ti servono le espressioni regolari

Immagina di dover controllare se un indirizzo email è scritto correttamente, o di dover trovare tutti i numeri di telefono in un testo. Potresti farlo con decine di condizioni `if`, oppure usare uno strumento molto più potente: le **espressioni regolari**.

Cosa sono le espressioni regolari?

Le **espressioni regolari** (dette anche "regex" o "regexp") sono una mini-lingua per descrivere **schemi di testo**. Servono a:

- Verificare se una stringa corrisponde a un certo formato ("è un'email valida?")
- Trovare schemi in un testo ("tutti i numeri in questo paragrafo")
- Estrarre informazioni ("dammi solo la data da questa riga")
- Sostituire parti di testo ("sostituisci tutti i numeri con `X` ")

In Python si usano con il modulo `re` :

I mattoncini di base: cosa significano i simboli

Simbolo	Significato	Esempio
<code>.</code>	Qualsiasi carattere (eccetto a capo)	<code>a.c</code> trova "abc", "aXc", "a3c"...
<code>\d</code>	Una cifra (0-9)	<code>\d\d</code> trova "42", "07"...
<code>\D</code>	Non una cifra	
<code>\w</code>	Lettera, cifra o underscore	
<code>\s</code>	Spazio, tab o a capo	
<code>^</code>	Inizio della stringa	<code>^Ciao</code> trova solo stringhe che iniziano con "Ciao"
<code>\$</code>	Fine della stringa	<code>fine\$</code> trova solo stringhe che terminano con "fine"
<code>[abc]</code>	Uno tra a, b, c	
<code>[a-z]</code>	Una lettera minuscola qualsiasi	
<code>[0-9]</code>	Una cifra qualsiasi (come <code>\d</code>)	

Quantificatori: quante volte?

Simbolo	Significato
<code>*</code>	Zero o più volte
<code>+</code>	Una o più volte
<code>?</code>	Zero o una volta
<code>{3}</code>	Esattamente 3 volte
<code>{2,5}</code>	Da 2 a 5 volte

Cercare uno schema in una stringa

`re.search()` — cerca ovunque nella stringa

```
testo = "Il mio numero è 334-1234567"

# Schema: 3 cifre, un trattino, 7 cifre
risultato = re.search(r"\d{3}-\d{7}", testo)

if risultato:
    print(f"Numero trovato: {risultato.group()}") # → 334-1234567
else:
    print("Nessun numero trovato")
```

Il prefisso `r` prima delle virgolette (`r"..."`) indica una **raw string** (stringa "grezza"). Senza la `r`, Python interpreterebbe `\d` come una sequenza speciale (come `\n` per andare a capo), stravolgendo il significato della regex. Con la `r`, Python lascia il testo esattamente come l'hai scritto, senza trasformare niente. Usa sempre `r"..."` quando scrivi espressioni regolari.

`re.match()` — cerca solo all'inizio della stringa

```
testo = "Ciao, mondo!"
risultato = re.match(r"Ciao", testo) # La stringa inizia con "Ciao"?

if risultato:
    print("Trovato!")
else:
    print("Non trovato.")
```

`re.findall()` — trova TUTTE le corrispondenze

```
testo = "Ho 3 mele, 5 banane e 2 arance."
numeri = re.findall(r"\d+", testo)  # Trova tutti i gruppi di cifre
print(numeri)  # ['3', '5', '2']
```

Sostituire testo: `re.sub()`

```
testo = "Il prezzo è 10 euro, poi 20 euro."
nuovo = re.sub(r"\d+", "X", testo)  # Sostituisce tutti i numeri con "X"
print(nuovo)  # Il prezzo è X euro, poi X euro.
```

Dividere testo: `re.split()`

Come `str.split()`, ma puoi dividere su uno schema complesso invece che su un singolo carattere:

```
testo = "mela; banana, uva  ciliegia"
# Divide su punto e virgola, virgola o spazi (anche multipli)
parti = re.split(r"[;,\s]+", testo)
print(parti)  # ['mela', 'banana', 'uva', 'ciliegia']
```

Estrarre parti specifiche: i gruppi con `()`

Le parentesi tonde creano dei **gruppi** — puoi estrarre separatamente ogni parte del risultato:

```
testo = "Data di nascita: 15/03/2008"
match = re.search(r"(\d{2})/(\d{2})/(\d{4})", testo)

if match:
    giorno = match.group(1)    # Primo gruppo → 15
    mese = match.group(2)      # Secondo gruppo → 03
    anno = match.group(3)      # Terzo gruppo → 2008
    print(f"Giorno: {giorno}, Mese: {mese}, Anno: {anno}")
```

Riutilizzare uno schema: `re.compile()`

Se usi lo stesso schema molte volte, "compilalo" una volta sola per rendere il codice più efficiente:

```
schema_numeri = re.compile(r"\d+")    # Compila lo schema una volta

frasi = ["Ho 3 mele", "Ho 10 banane", "nessun numero qui"]
for frase in frasi:
    trovati = schema_numeri.findall(frase)
    if trovati:
        print(f"Trovati: {trovati}")
```


Esempi pratici

Verificare se un'email è valida

```
def valida_email(email):
    # Schema: caratteri@caratteri.dominio (almeno 2 lettere)
    schema = r"^[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}$"
    return bool(re.match(schema, email))

print(valida_email("alice@esempio.it"))    # True ← valida
print(valida_email("non_valida.com"))      # False ← manca la @
print(valida_email("@dominio.it"))         # False ← manca la parte prima
della @
```

Verificare un numero di telefono italiano

```
def valida_telefono(numero):
    # Schema: opzionalmente +39, poi 9 o 10 cifre
    schema = r"^(\+39)?[0-9]{9,10}$"
    return bool(re.match(schema, numero))

print(valida_telefono("3341234567"))      # True
print(valida_telefono("+393341234567"))   # True
print(valida_telefono("abc"))             # False
```